

IRSDA: An Agent-Orchestrated Framework for Enterprise Intrusion Response

IRSDA: Intrusion Response System Digital Assistant

Paper: *IRSDA: An Agent-Orchestrated Framework for Enterprise Intrusion Response*
Autori: Damodar Panigrahi, Raj Patel, Shaswata Mitra, Sudip Mittal, Shahram Rahimi
Tipo: preprint arXiv, 24 novembre 2025

Il paper presenta principalmente un framework architetturale e un case study/proof-of-concept. Non propone una valutazione quantitativa estesa con benchmark, baseline e metriche complete.

1. Idea centrale del paper - Problema

Divario tra intrusion detection e intrusion response.

Gli IDS moderni possono utilizzare ML per ottenere detection più adattive, mentre la risposta all'incidente rimane spesso affidata a playbook statici e all'intervento umano.

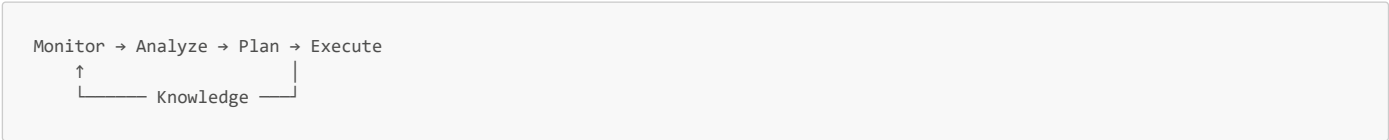
Conseguenza: ritardi e difficile reazione ad attacchi dinamici, distribuiti e multi-stage.

IRSDA vuole collegare detection e response attraverso un framework autonomo e policy-compliant. L'architettura combina:

- Self-Adaptive Autonomic Computing Systems (SA-ACS);
- paradigma MAPE-K;
- agentic AI;
- partition-specific agents;
- property Knowledge Graph;
- vector retrieval;
- Graph Retrieval-Augmented Generation (GRAG);
- cybersecurity-tuned LLM;
- Rules of Engagement (ROE);
- eventuali computational models, come GNN e reinforcement learning.

L'idea fondamentale è che il **LLM non venga lasciato libero di decidere sulla base della sola conoscenza parametrica**: viene invece inserito in una pipeline dove stato enterprise, relazioni infrastrutturali e policy **vengono recuperati dal Knowledge Graph**.

2. Loop MAPE-K



Le cinque fasi sono:

- Monitor: acquisizione di informazioni sullo stato dell'infrastruttura tramite log, telemetria ed eventi.
- Analyze: interpretazione dello stato del sistema e identificazione di anomalie/incidente.
- Plan: individuazione della risposta appropriata considerando anche le policy enterprise.
- Execute: esecuzione dell'azione di risposta.
- Knowledge: base informativa utilizzata dal ciclo. In questo paper viene modellata tramite un **Knowledge Graph enterprise-specifico**, integrato con indici vettoriali.

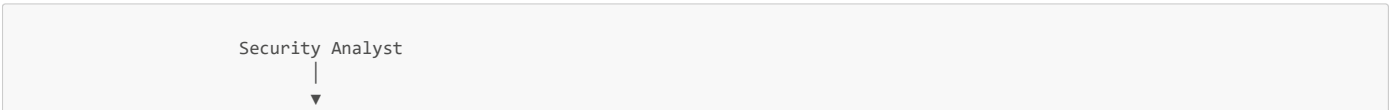
IRSDA porta il paradigma MAPE-K verso una cyber defense agentica, in cui il reasoning viene svolto da LLM e computational models, mentre il coordinamento è affidato a un sistema multi-agent e la conoscenza è fornita dal Knowledge Graph.

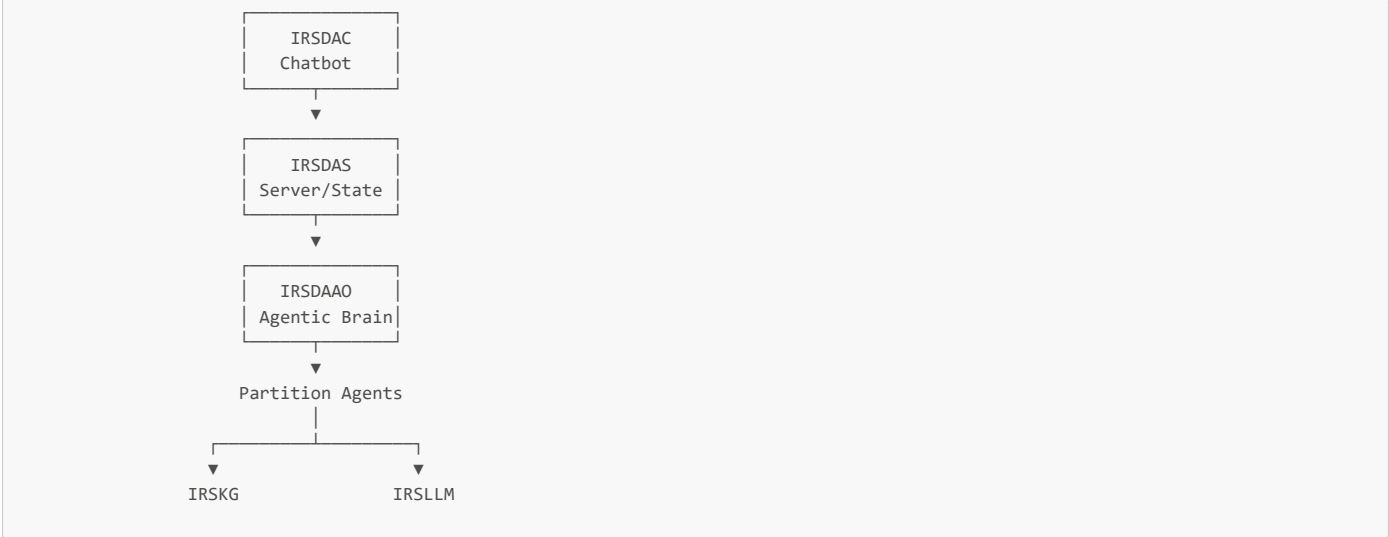
3. Architettura n-tier

IRSDA combina:

- **N-tier architecture**;
- **Client-Server (CS)**;
- **Multi-Agent System (MAS)**.

L'architettura è organizzata in cinque livelli.





Tier I — IRSDAC - Chatbot

Interfaccia web utilizzata dal security analyst per interrogare il sistema tramite linguaggio naturale.

Tier II — IRSDAS - Server

Gestisce:

- richieste client;
- sessioni;
- stato;
- coordinamento delle attività di response.

Tier III — IRSDAAO - Agentic Orchestration

È il livello agentic. Contiene l'**Agentic Brain (AB)**, che coordina gli agenti specializzati.

Tier IV — Agents

Gli agenti operano su specifiche partizioni del sistema enterprise.

Tier V — KG + LLM

Contiene:

- **IRSKG**, Knowledge Graph enterprise, contiene log, configurazioni e ROE;
- **IRSLLM**, LLM specializzato in cybersecurity, adattato su dataset pubblici ed enterprise-specifici.

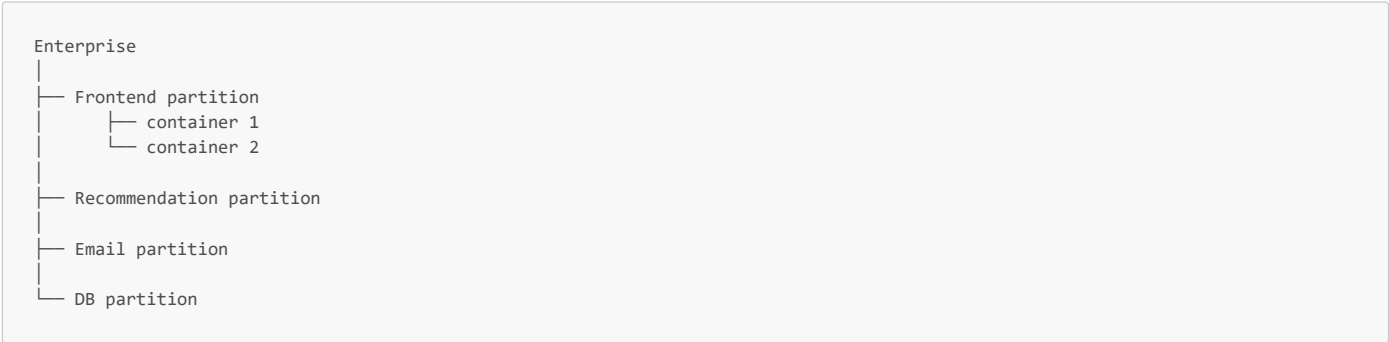
4. Partitioning dell'infrastruttura

L'intero sistema enterprise viene rappresentato come insieme di partizioni:

$$M = \{C_1, C_2, \dots, C_n\}$$

dove ogni C_i rappresenta una partizione associata a un determinato tipo di componente.

Esempio:



Vantaggio: possibilità di associare agenti differenti alle singole partizioni, riducendo la complessità del reasoning locale, permettendo successivamente una coordinazione globale.

5. IRS Knowledge Graph (IRSKG)

IRSKG rappresenta l'intera infrastruttura enterprise come un **property graph**:

$$G = (V, E)$$

dove:

- V = insieme dei vertici;
- E = insieme degli archi.

Ogni vertice è modellato come:

$$V_i = \{L(V_i), P(V_i)\}$$

dove:

- $L(V_i)$ = label;
- $P(V_i)$ = dizionario delle proprietà.

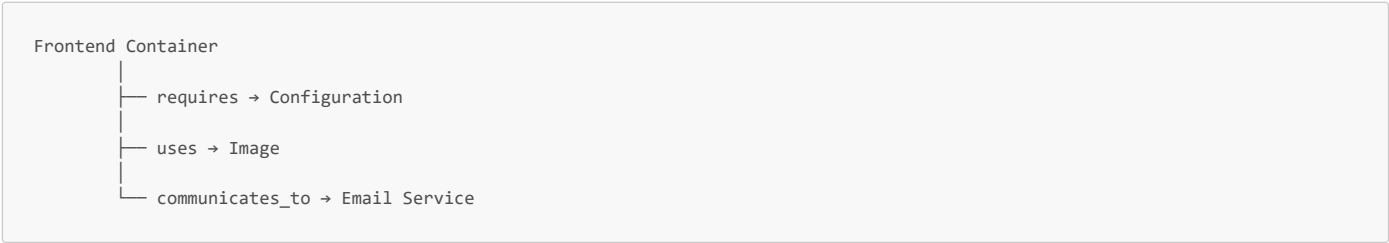
Ogni edge:

$$E_{i,j} = \{L(E_{i,j}), P(E_{i,j})\}$$

Vantaggio: questa modellazione consente di rappresentare sia gli attributi delle entità sia le relazioni tra esse.

Migliore rispetto a un semplice repository documentale che non esplicita relazioni tra asset.

Esempio concettuale:



6. Informazioni memorizzate in IRSKG

6.1 System configurations

Le componenti enterprise vengono modellate come tipi, partizioni e istanze.

Nel case study del paper viene mostrato un frontend partition con componenti container e una configurazione associata.

6.2 Enterprise logs

Gli eventi di rete vengono trasformati in entità e relazioni del grafo.

Esempio di TCP handshake:

```
192.168.1.100 -> 192.168.1.101 : SYN
192.168.1.101 -> 192.168.1.100 : SYN-ACK
192.168.1.100 -> 192.168.1.101 : ACK
192.168.1.101 -> 192.168.1.100 : ACK
```

Il paper utilizza Graylog per la raccolta dei log e mostra come tali eventi vengano rappresentati in Neo4j mediante nodi IP e relazioni timestamped.

7. Feature engineering per computational models

Il KG viene utilizzato anche come base per modelli ML, in particolare GNN.

Gli autori introducono una feature **count** per i vertici:

$$P_{count}(V_i) = deg(V_i)$$

dove $deg(V_i)$ è il grado del nodo.

Per gli edge:

$$P_{count}(E_{i,j}) = P_{count}(V_i) + P_{count}(V_j)$$

Nel case study del papaer, i nodi relativi agli IP 192.168.1.100 e 192.168.1.101 hanno count pari a 2, mentre l'edge tra i due conserva una feature di count derivata dalla trasformazione definita dal paper.

Quindi IRSKG non è solamente una memoria per LLM: è anche una rappresentazione strutturata utilizzabile da modelli computazionali.

8. Rules of Engagement (ROE)

Le **Rules of Engagement** sono uno degli elementi più importanti del framework.

Codificano:

- condizioni di attivazione della response;
- azioni vietate;
- vincoli di sicurezza;
- policy operative dell'organizzazione.

Gli autori definiscono $R = \{R_i\}$ e modellano una regola come:

$$R_i = \{V_a(R_i), E(R_i), V_b(R_i)\}$$

cioè una relazione tra due vertici con proprietà e constraint espliciti.

Nel caso di studio:

```
frontend-service
  |
  | COMMUNICATES_TO
  ▼
email-service

action = SYN
constraint = DENY
# tale regola vieta al frontend-service di stabilire una comunicazione SYN con l'email-service.
```

Significato architetturale

Questo introduce una separazione tra:

```
LLM reasoning
  ↓
"Quale risposta sembra appropriata?"
```

e:

```
ROE / policy
  ↓
"Quali azioni sono consentite?"
```

Il LLM viene inserito in un sistema in cui le policy enterprise devono limitare le azioni ammissibili.

9. IRSLLM

IRSDA utilizza un LLM specializzato in cybersecurity denominato:

IRSLLM — Intrusion Response System Security Large Language Model

```
Regular Enterprise LLM
  ↓
CyBERT
  ↓
CyBERTuned
  ↓
+ enterprise-specific data
  ↓
LoRA
  ↓
IRSLLM
```

Il modello parte da conoscenza cybersecurity e viene poi adattato all'organizzazione tramite fine-tuning su corpus derivati dall'IRSKG, utilizzando **Low-Rank Adaptation (LoRA)**.

10. Perché fine-tuning + RAG

Il fine-tuning serve soprattutto a fornire **domain adaptation**.

Non è sufficiente per conoscere lo stato attuale dell'infrastruttura, ad esempio:

- quali servizi sono compromessi;
- quali IP stanno comunicando;
- quale configurazione è attiva;
- quali ROE sono correntemente valide.

Queste informazioni sono dinamiche.

Il ruolo dei due meccanismi è quindi:

Fine-tuning → Cybersecurity/enterprise domain adaptation

RAG → Current enterprise context

Vantaggio: riduzione delle hallucination e aumento factual fidelity.

11. Prompt whitelisting

IRSLLM non viene trattato come un agente generico capace di eseguire qualsiasi task.

Il framework limita il modello a due compiti principali:

1. **Question Answering**
2. **Summarization**

Il controllo avviene attraverso **prompt whitelisting**.

```

User Query
  ↓
Whitelisted task/template
  ↓
Retrieved context
  ↓
LLM
  
```

anziché:

```

User Query
  ↓
LLM
  ↓
arbitrary behavior
  
```

Vantaggio: mitigazione del rischio di prompt injection.

12. Prompt templates

Il framework mantiene un insieme: $T = \{T_1, T_2, \dots, T_n\}$ di template.

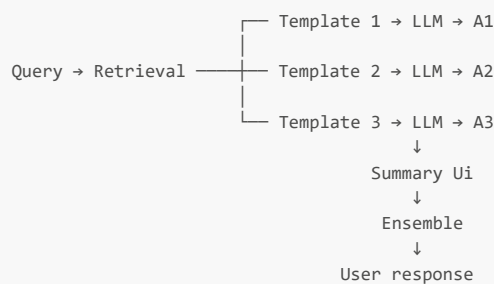
Le tecniche considerate includono:

- zero-shot;
- multi-shot;
- chain-of-thought.

Per una query Q_i , il sistema seleziona un template T_i , lo arricchisce con il contesto recuperato e ottiene:

$$A_i = IRSLLM(T_i, Q_i, Context)$$

Le risposte vengono successivamente sintetizzate in: U_i e aggregate in un insieme di candidate answers.



13. Graph Retrieval-Augmented Generation (GRAG)

IRSDA utilizza **Graph Retrieval-Augmented Generation**, indicato come GRAG.

Per una query Q_i :

$$Q_i \rightarrow \text{Retrieve}(\text{IRSKG}, Q_i) \rightarrow T_i(\text{Context}, Q_i) \rightarrow \text{IRSLLM} \rightarrow A_i$$

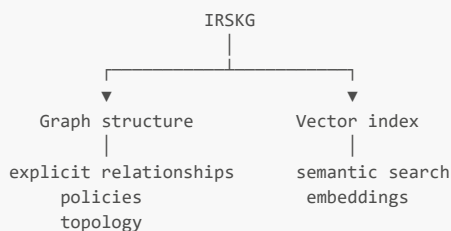
Per la summarization:

$$A_i + \text{RetrievedContext} \rightarrow \text{IRSLLM} \rightarrow U_i$$

Gli indici vettoriali vengono creati durante l'ingestion e mantenuti all'interno dell'infrastruttura di IRSKG per supportare il retrieval semantico.

14. Graph + Vector Retrieval

IRSKG ha quindi due funzioni complementari:



Nel case study il paper mostra un esempio basato su Neo4j e LangChain:

```

Neo4jVector.from_existing_graph(
    embedding=OpenAIEmbeddings(),
    ...
)
  
```

e una query:

```

similarity_search(
    "frontend partition configuration"
)
  
```

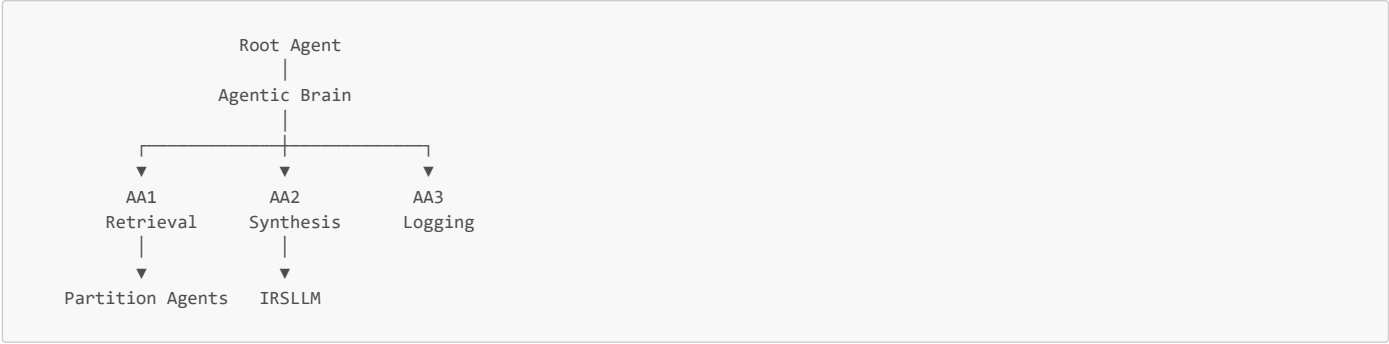
per recuperare semanticamente la configurazione associata.

Questa combinazione permette di unire *Semantic Retrieval* + *Explicit Relationships*.

15. Multi-Agent System e Agentic Brain

L'orchestrazione è basata su un **Agentic Brain (AB)** che coordina sub-agent specializzati. Nell'implementazione attuale sono presenti tre agenti:

- AA1: gestisce la coordinazione degli IRS Agents e il retrieval.
- AA2: sintetizza i risultati e interagisce con IRSLLM.
- AA3: monitora e registra telemetry/metadata.



Il valore dell'approccio multi-agent deriva quindi dalla separazione funzionale dei compiti:

Retrieval ≠ Synthesis ≠ Monitoring

16. Control flow completo

1. Query

L'analista invia una richiesta tramite IRSDAC.

"Is there active breach in the system?"

2. IRSDAS

Il server riceve la richiesta e la trasferisce all'orchestrator.

3. Agentic Brain

L'AB assegna il task ad AA1.

4. Partition-specific retrieval - AA1

AA1 coordina gli agenti associati alle diverse partizioni.

5. Retrieval

Ogni agente interroga gli indici vettoriali dell'IRSKG.

6. Computational models

Gli agenti possono utilizzare modelli computazionali per valutare possibili azioni.

7. Prompt augmentation

Le evidenze recuperate e i risultati computazionali vengono inseriti nel prompt.

8. IRSLLM

Il modello genera le risposte.

9. Aggregazione

AA1 restituisce i risultati all'AB.

10. Sintesi - AA2

L'AB attiva AA2, che sintetizza i risultati.

11. Logging - AA3

AA3 opera asincronamente per raccogliere telemetria e metadata.

12. Presentation

La risposta viene formattata in HTML a tab multipli e restituita all'analista.

17. Stateful agent orchestration

Il sistema mantiene il contesto tramite `session.state`, consentendo di preservare stato tra interazioni.

AA3 viene eseguito asincronamente per il logging operativo, aumentando la tracciabilità del workflow.

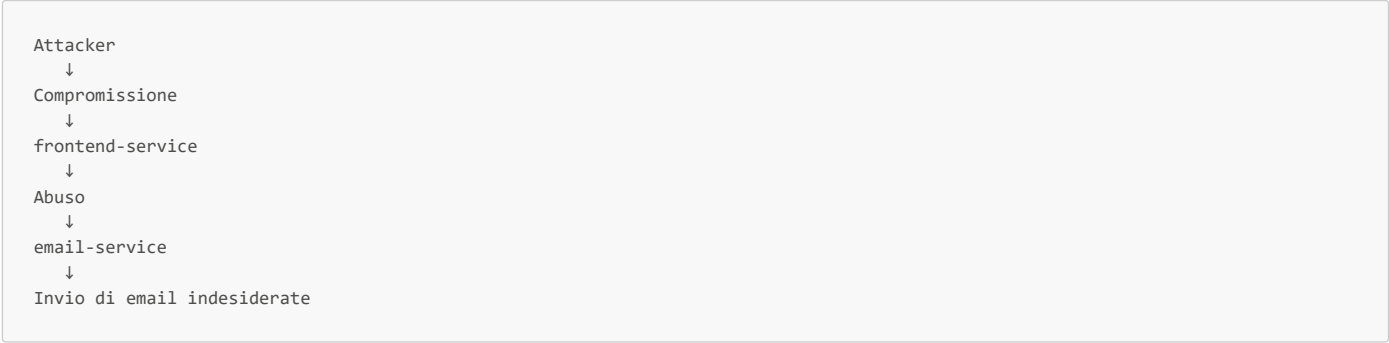
Questa caratteristica è rilevante perché IRSDA non tratta necessariamente ogni query come un evento completamente isolato.

18. Caso di studio: Google Online Boutique

Il framework viene dimostrato usando **Online Boutique 2.0**, applicazione e-commerce open-source basata su microservizi e composta da **11 servizi**.

19. Scenario di attacco

Scenario ipotizzato:



IRSDA ha due obiettivi:

- 1. impedire ulteriori email malevole;
- 2. ripristinare il frontend in uno stato sicuro.

Per semplicità gli autori si concentrano sulla parte di containment/prevention e assumono una ROE che vieta la comunicazione tra il frontend compromesso e il servizio target. `filecite\turn2file1\L606-L621`

20. Esempio di reasoning end-to-end

L'analista chiede:

"Is there active breach in the system?"

Il sistema recupera evidenze dalle varie partizioni e produce una risposta del tipo:

- 1. frontend-service is compromised.
- 2. frontend-service is attempting to connect to email-service, which is prohibited by the admin.

L'analista può quindi chiedere:

"Describe the rule that prohibits the connection."

IRSDA recupera la ROE R1 e la restituisce insieme alla relativa rappresentazione. `filecite\turn2file1\L836-L855`

La catena è quindi:

$$Query \rightarrow Retrieval \rightarrow Policy \rightarrow LLM \rightarrow Explanation$$

21. Detection, reasoning, decision e enforcement

Per una lettura critica è utile distinguere:

Detection

Identificazione del breach attraverso telemetry/log e stato del sistema.

Reasoning

LLM + retrieval + computational models.

Decision

Scelta dell'azione nel rispetto delle ROE.

Enforcement

Applicazione effettiva dell'azione sull'infrastruttura.

Il paper descrive chiaramente detection, reasoning, policy alignment e containment a livello di caso di studio, ma non presenta un meccanismo generalizzato di enforcement operativo dettagliato, come un controller firewall/API, EDR o Kubernetes policy engine che rappresenti l'intero percorso fino all'applicazione della policy.

Questo è un limite importante da evidenziare in una tesi.

22. Google Agent Development Kit

Il prototipo IRSDAAO utilizza il **Google Agent Development Kit (ADK)**.

Gli autori lo scelgono per:

- supporto all'orchestrazione;
- capacità multimodali;
- sandbox;
- riduzione di dipendenze;
- supporto allo sviluppo di workflow agentici.

L'architettura, tuttavia, viene presentata come platform-agnostic. `filecite[turn2file1][L574-L595]`

Nel codice mostrato compare:

```
root_agent = Agent(
    tools=[
        AgentTool(agent=AA1),
        AgentTool(agent=AA2),
        AgentTool(agent=AA3)
    ]
)
```

e viene utilizzato anche:

```
InMemorySessionService()
```

per la gestione delle sessioni. `filecite[turn2file1][L906-L931]`

Nel prototipo viene inoltre indicato:

```
GEMINI_MODEL = "gemini-2.0-flash"
```

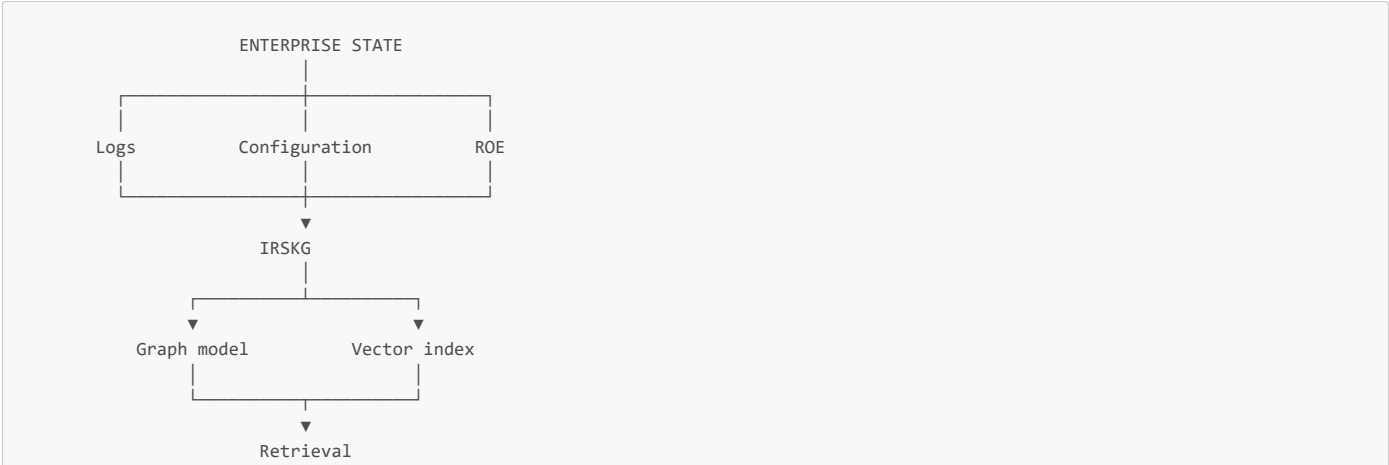
È quindi importante distinguere:

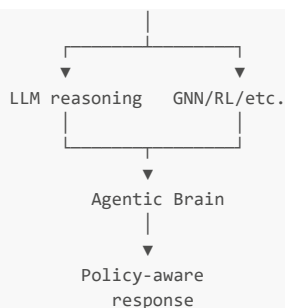
- **IRSLLM**, modello enterprise cybersecurity-oriented;
- **modello impiegato nell'implementazione dell'orchestration layer**.

23. Architettura AI ibrida

Uno degli aspetti più interessanti del lavoro è l'integrazione di componenti differenti:

$$Knowledge\ Graph + Vector\ Retrieval + LLM + Multi - Agent + Computational\ Models + ROE$$





Una possibile interpretazione teorica è quella di una architettura **neuro-symbolic-like**:

- componente neurale/probabilistica → LLM + modelli ML;
- componente strutturata/simbolica → Knowledge Graph + ROE;
- componente di controllo → Agentic Brain + agents.

Questa etichetta va però presentata come interpretazione architetturale e non come classificazione esplicita degli autori.

24. Mitigazione delle hallucination

Il paper combina diversi meccanismi:

Domain fine-tuning

Specializzazione cybersecurity/enterprise.

GRAG/RAG

Grounding su dati reali.

Prompt templates

Definizione esplicita del task.

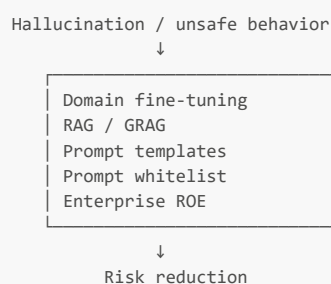
Prompt whitelisting

Riduzione dello spazio comportamentale del modello.

ROE

Vincolo sulle azioni possibili.

Schema:



Gli autori dichiarano esplicitamente che grounding e prompt design vengono utilizzati per migliorare la factual relevance e ridurre hallucination.

📄filecite📄turn2file0📄L402-L423📄 📄filecite📄turn2file1📄L524-L571📄

25. Principale limite sperimentale

Il paper non fornisce una valutazione quantitativa estesa con:

- precision;
- recall;
- F1;
- false positive rate;
- false negative rate;
- response latency;
- MTTR;
- retrieval precision/recall;

- faithfulness;
- policy violation rate;
- agent success rate;
- token/cost analysis;
- confronto sistematico con baseline;
- ablation Graph RAG vs RAG tradizionale.

Il lavoro dimostra soprattutto la **fattibilità architetturale tramite un case study**.

Per questo, nella tesi è più corretto descriverlo come:

framework + architectural demonstration + proof-of-concept/case study

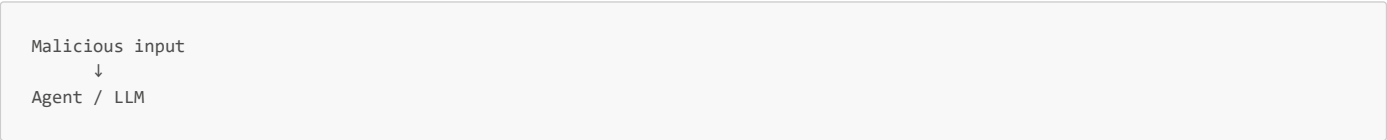
e non come dimostrazione quantitativa della superiorità di IRSDA rispetto a un baseline.

26. Limiti di sicurezza del framework

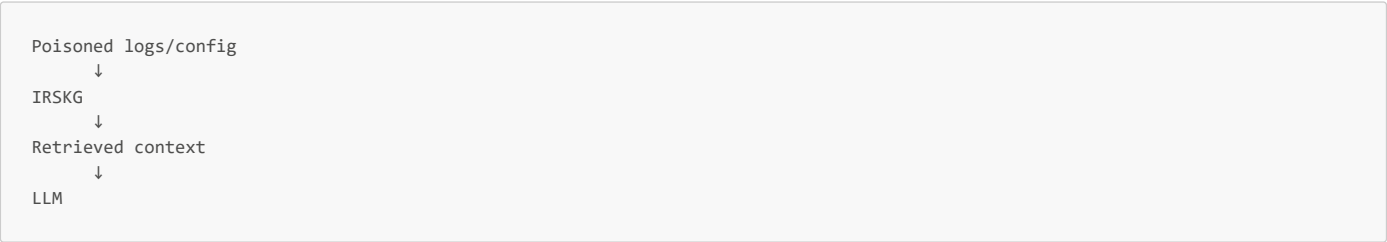
Gli stessi autori riconoscono che i componenti ML possono essere compromessi da un attaccante e dichiarano di non affrontare tali scenari nell'ambito del paper.

L'architettura permette di individuare diverse potenziali superfici d'attacco:

Prompt injection



Knowledge Graph poisoning



Retrieval manipulation

Un attaccante potrebbe tentare di manipolare il contenuto recuperato dal sistema.

Agent hijacking

Un sub-agent compromesso potrebbe produrre un output errato che viene poi propagato dall'Agentic Brain.

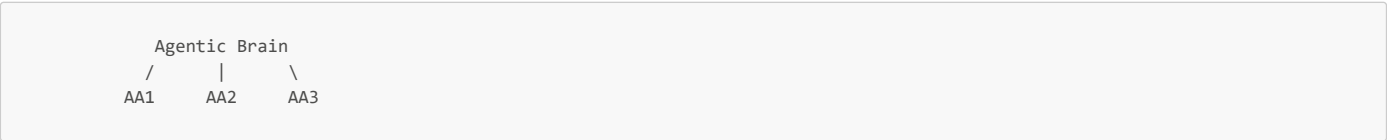
Tool abuse

L'integrazione con strumenti esterni aumenta la superficie di attacco e il rischio di azioni non autorizzate.

Questi scenari sono deduzioni dall'architettura e **non risultati sperimentali presentati dal paper**.

27. Possibile single point of failure: Agentic Brain

L'Agentic Brain rappresenta un coordinatore centrale:



Sebbene l'esecuzione sia distribuita, il coordinamento rimane centralizzato.

Questo può creare potenzialmente:

- single point of failure;
- single point of compromise;
- colli di bottiglia;
- dipendenza dal corretto routing centrale.

Il paper non presenta un'analisi quantitativa di fault tolerance o resilienza dell'Agentic Brain.

Possibile innovazione per tesi.

28. Future work

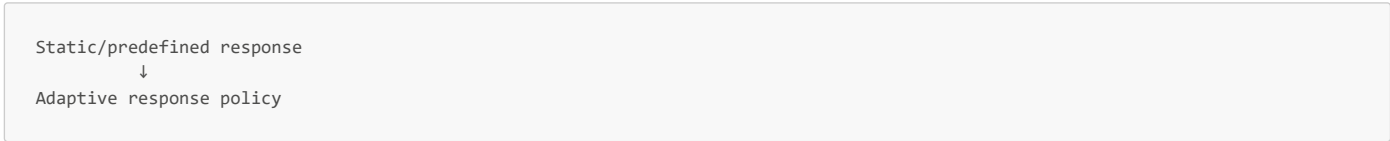
La principale direzione futura indicata è l'integrazione del **Reinforcement Learning** negli IRS Agents.
L'obiettivo è permettere agli agenti di apprendere politiche di risposta adattive nel tempo:

$$Policy(s_t) \rightarrow a_t$$

considerando:

- evoluzione degli attack pattern;
- feedback ambientale;
- safety constraints;
- policy aziendali.

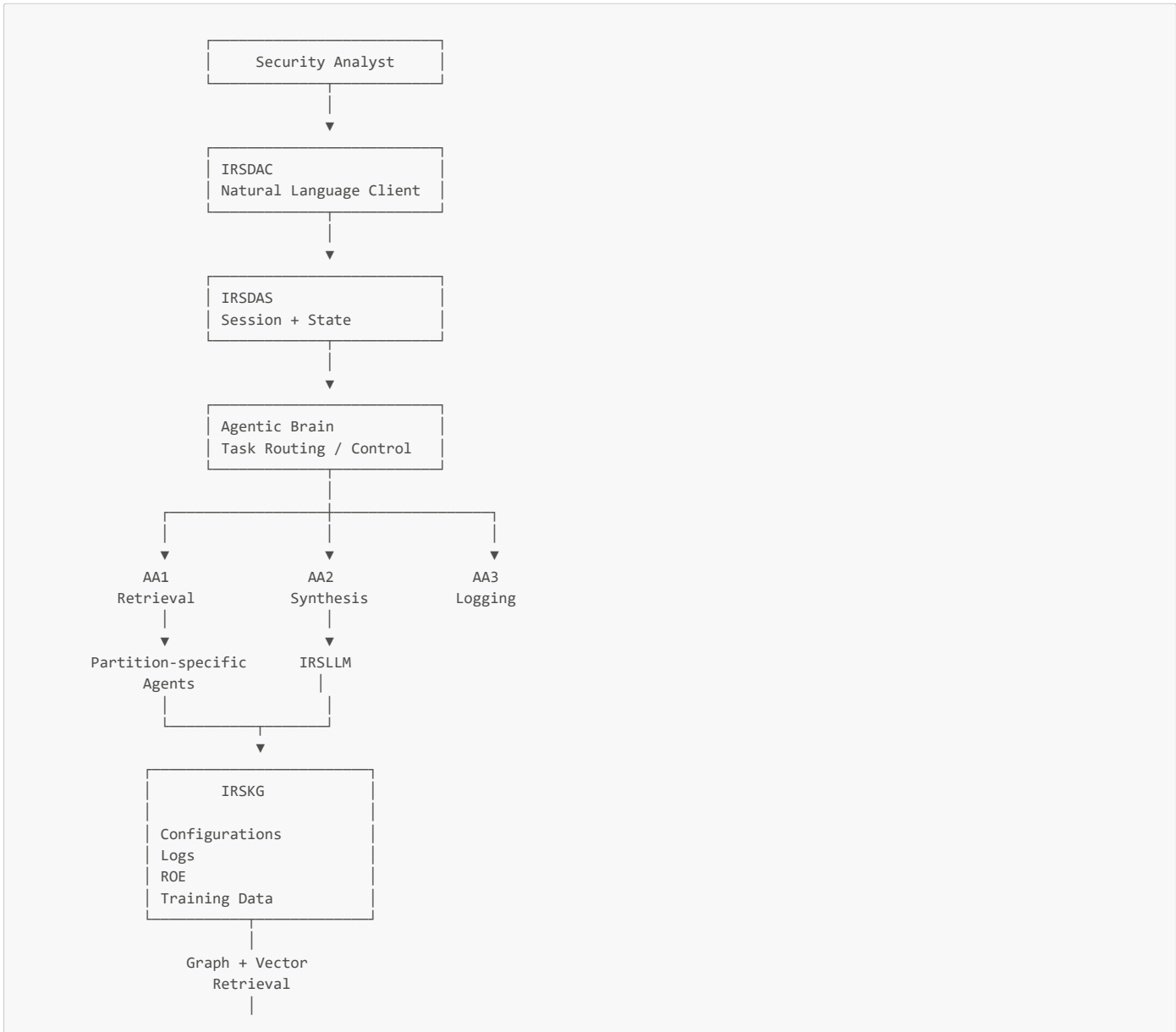
Il passaggio proposto è quindi:



mantenendo comunque i vincoli definiti dalle ROE.

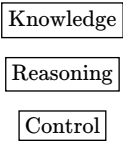
29. Il modello concettuale complessivo

Per una tesi, IRSDA può essere rappresentato così:





L'architettura può essere interpretata attraverso tre piani:



dove:

- **Knowledge** → IRSKG
- **Reasoning** → IRSLLM + computational models
- **Control** → Agentic Brain + IRS Agents + ROE

Questa è un'interpretazione sintetica dell'architettura proposta.

30. Cosa è realmente innovativo

Il contributo non consiste nell'invenzione individuale di LLM, RAG, LoRA, Knowledge Graph, MAS o MAPE-K.

Il valore è nella loro integrazione:

$$SA-ACS + MAPE-K + Partitioned IRS + Knowledge Graph + Vector Retrieval + GRAG + Cybersecurity LLM + Prompt Whitelisting$$

con l'obiettivo di ottenere una **intrusion response autonoma ma policy-constrained**. Le tre contribuzioni dichiarate dagli autori sono il framework IRSDA, il Graph RAG allineato con telemetry e policies e il case study end-to-end su una microservice application. [filecite\turn2file0\L70-L94](#)

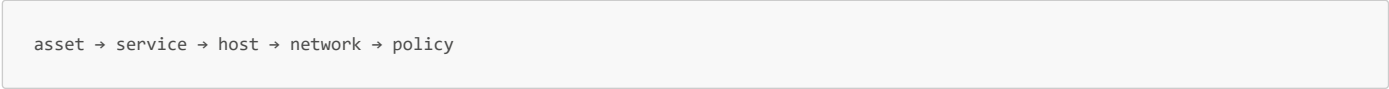
31. Aspetti particolarmente utili per la tesi

LLM ≠ decision engine

L'LLM svolge il reasoning, ma viene grounded e constrained.

RAG non è sufficiente da solo

L'ambiente enterprise contiene relazioni strutturali:



che un Knowledge Graph rappresenta esplicitamente.

Fine-tuning e RAG hanno ruoli differenti

$$Fine-tuning \rightarrow \text{domain knowledge}$$
$$RAG \rightarrow \text{current enterprise state}$$

Agentic orchestration separa le responsabilità

$$Retrieval \neq Synthesis \neq Monitoring$$

Le policy devono vincolare il reasoning

Una formalizzazione concettuale utile è:

$LLM\ output \in AllowedActions(ROE)$

Questa formula non è esplicitamente definita nel paper, ma rappresenta bene il principio architetturale.

Explainability e provenance

Le risposte possono essere ricondotte a:

```
log
configuration
graph relation
ROE
```

e quindi non dipendono soltanto dalla conoscenza parametrica dell'LLM.

32. Valutazione critica

Aspetto	Valutazione
Idea architetturale	Molto interessante
Integrazione AI/cybersecurity	Molto forte
Uso del Knowledge Graph	Particolarmente interessante
GRAG/RAG	Ben integrato nel design
Multi-agent orchestration	Interessante
Policy enforcement	Concettualmente forte, implementazione poco dettagliata
Explainability	Buona a livello architetturale
Validazione quantitativa	Debole/limitata
Scalabilità dimostrata	Non sufficientemente valutata
Security del framework stesso	Non analizzata in profondità
Robustezza contro prompt injection	Non valutata sperimentalmente
Data/Knowledge poisoning	Non valutato
Generalizzazione oltre Online Boutique	Non dimostrata

La debolezza principale è il divario tra **architettura proposta** e **validazione sperimentale**. Il paper dimostra bene come potrebbe funzionare un sistema di intrusion response agentico, ma molto meno quanto sia migliore rispetto ai sistemi esistenti.

33. Formulazione pronta per la tesi

Panigrahi et al. propongono IRSDA, un framework per l'automazione dell'intrusion response enterprise che combina il paradigma MAPE-K dei sistemi self-adaptive con una multi-agent orchestration architecture, un property Knowledge Graph contenente stato, configurazioni, log e Rules of Engagement, e un Large Language Model specializzato in cybersecurity. Il modello viene adattato al contesto enterprise tramite LoRA e integrato con un meccanismo Graph Retrieval-Augmented Generation, che fornisce al modello evidenza contestuale proveniente dall'infrastruttura e dalle policy aziendali. L'architettura è dimostrata tramite il caso di studio Google Online Boutique, nel quale il sistema rileva una compromissione del frontend e identifica una violazione di policy relativa alla comunicazione con il servizio email.

Una precisazione metodologica utile:

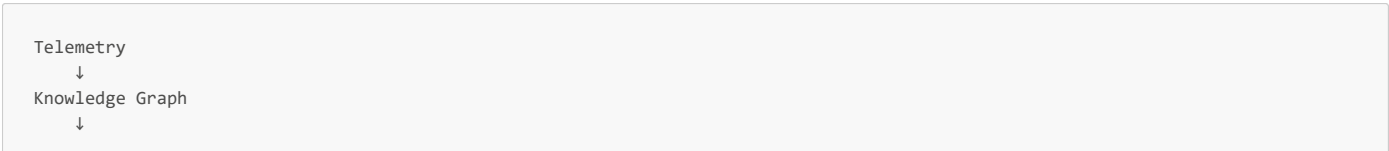
Il lavoro costituisce principalmente una dimostrazione architetturale e un proof-of-concept, piuttosto che una valutazione comparativa quantitativa su larga scala; inoltre, gli stessi autori lasciano aperto il problema della sicurezza degli stessi componenti ML e agentici contro attacchi adversariali.

34. Sintesi finale

Il paper propone di trasformare l'Intrusion Response da:

```
IDS → Alert → Human → Playbook
```

a:



```
graph TD; A[Partition-specific Agents] --> B[GRAG]; B --> C[Cybersecurity LLM]; C --> D[Policy / ROE constraints]; D --> E[Agentic orchestration]; E --> F[Automated response]; F --> G[Explanation + Audit trail];
```

La vera idea chiave è quindi **non utilizzare un LLM come componente autonoma e incontrollata**, ma inserirlo in un sistema **knowledge-grounded, partition-aware, multi-agent e policy-constrained**. Questo rende IRSDA particolarmente rilevante per una tesi su **Agentic AI applicata alla cybersecurity**.